COMPTE RENDU DE TRAVAUX PRATIQUES

Programmation pour cible embarquée

TP3 — Commande d’un feu tricolore par accès direct aux registres

Étudiants	Yousri Elouej, Anis Boubakri, Mohamed Amine Bouguila
Classe	LISI 3
Microcontrôleur	STM32F407VG — ARM Cortex-M4
Environnement	Keil µVision / MDK-ARM
Langage et méthode	Langage C — accès direct aux registres CMSIS



Objectif du TP. Configurer PA5, PA6 et PA7 du port GPIOA comme sorties numériques, puis commander successivement les LED rouge, jaune et verte au moyen de temporisations logicielles dans une boucle infinie.

Année universitaire : 2026–2027

1. Objectif et principe de fonctionnement

Ce TP vise à programmer une séquence de feu tricolore sur le microcontrôleur STM32F407VG, sans utiliser les fonctions HAL de commande des GPIO. Le programme active l'horloge du port GPIOA, configure trois broches en sortie, puis écrit successivement les valeurs correspondantes dans le registre de sortie ODR.

La séquence réellement définie par le code fourni est la suivante :

Étape	Broche	LED	Attente
1	PA5	Rouge	9 000 000 itérations
2	PA6	Jaune	4 000 000 itérations
3	PA7	Verte	9 000 000 itérations
4	PA6	Jaune	4 000 000 itérations

Après la quatrième étape, la boucle recommence à l'étape 1. Cette séquence rouge → jaune → vert → jaune est celle du programme fourni ; elle n'est pas modifiée dans ce compte rendu.

2. Matériel et environnement de travail

- Carte équipée d'un microcontrôleur STM32F407VG.
- Ordinateur avec Keil µVision / MDK-ARM et chaîne de compilation ARM.
- Breadboard, trois LED (rouge, jaune et verte), résistances de limitation de courant et fils de connexion.
- Câble USB et interface de programmation/débogage compatible avec la carte.

Chaque LED doit être protégée par une résistance montée en série. Le raccordement exact des LED aux broches PA5, PA6 et PA7 doit être contrôlé sur le montage réel : la photographie montre les composants et les fils, mais ne permet pas de certifier toutes les connexions électriques.

2.1. Montage électrique sur breadboard

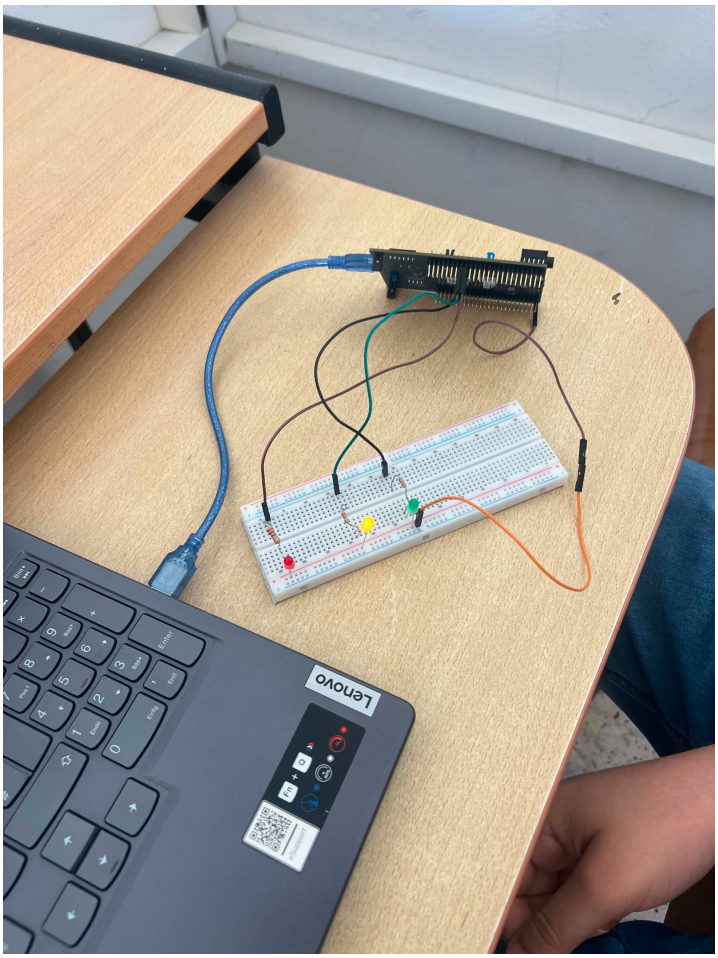


Figure 1 — Montage expérimental : carte STM32 reliée par USB, breadboard et trois LED.

3. Programme C utilisé

```
#include "stm32f407xx.h"

void delay(volatile uint32_t time)
{
    while (time--)
    {
        __NOP();
    }
}

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

    /* PA5 rouge, PA6 jaune, PA7 vert */
    GPIOA->MODER &= ~(3U << 10);
    GPIOA->MODER &= ~(3U << 12);
    GPIOA->MODER &= ~(3U << 14);

    GPIOA->MODER |= (1U << 10);
    GPIOA->MODER |= (1U << 12);
    GPIOA->MODER |= (1U << 14);

    while (1)
    {
        /* Rouge */
        GPIOA->ODR = (1U << 5);
        delay(9000000);

        /* Jaune */
        GPIOA->ODR = (1U << 6);
        delay(4000000);

        /* Vert */
        GPIOA->ODR = (1U << 7);
        delay(9000000);

        /* Jaune */
        GPIOA->ODR = (1U << 6);
        delay(4000000);
    }
}
```

Le code est reproduit conformément à la version fournie. L'en-tête stm32f407xx.h doit être disponible dans le projet CMSIS configuré pour le STM32F407VG.

4. Explication du programme

#include "stm32f407xx.h"

Inclut les définitions CMSIS du STM32F407VG : adresses des registres, structures des périphériques et constantes symboliques.

void delay(volatile uint32_t time)

Déclare une fonction d'attente qui reçoit un entier non signé de 32 bits. Le mot-clé volatile limite certaines optimisations de la variable par le compilateur ; il ne garantit pas une durée précise.

while (time--)

Répète la boucle tant que la valeur précédente de time n'est pas nulle, en décrémentant la variable à chaque passage.

__NOP();

Exécute une instruction « No Operation ». Elle ne modifie pas directement les données du programme et sert ici à occuper du temps processeur.

int main(void)

Définit le point d'entrée du programme C. void signifie qu'aucun argument n'est transmis à la fonction.

RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

Active l'horloge du port GPIOA. L'opérateur |= met à 1 le bit GPIOAEN sans modifier les autres bits du registre.

GPIOA->MODER &= ~(3U << 10);

Efface les deux bits MODER[11:10], qui définissent le mode de PA5. Le champ est ainsi remis à 00 avant sa configuration.

GPIOA->MODER &= ~(3U << 12);

Efface les deux bits de configuration du mode de PA6, MODER[13:12].

GPIOA->MODER &= ~(3U << 14);

Efface les deux bits de configuration du mode de PA7, MODER[15:14].

GPIOA->MODER |= (1U << 10);

Place le champ MODER de PA5 à 01 : PA5 est configurée en sortie générale.

GPIOA->MODER |= (1U << 12);

Place le champ MODER de PA6 à 01 : PA6 est configurée en sortie générale.

GPIOA->MODER |= (1U << 14);

Place le champ MODER de PA7 à 01 : PA7 est configurée en sortie générale.

while (1)

Crée une boucle infinie afin que le cycle des LED se répète continuellement.

GPIOA->ODR = (1U << 5);

Écrit un 1 sur le bit 5 de l’ODR et des 0 sur les autres bits du registre. PA5 est donc la seule sortie mise à l’état haut par cette affectation.

delay(9000000);

Exécute neuf millions d’itérations de l’attente logicielle avant de passer à l’étape suivante. Ce nombre ne représente pas directement une durée en secondes.

GPIOA->ODR = (1U << 6);

Met PA6 à l’état haut et écrit 0 dans les autres bits de l’ODR ; la LED jaune est sélectionnée si le câblage correspond au commentaire du programme.

GPIOA->ODR = (1U << 7);

Met PA7 à l’état haut et écrit 0 dans les autres bits de l’ODR ; la LED verte est sélectionnée selon le câblage prévu.

delay(4000000);

Exécute quatre millions d’itérations, soit un comptage inférieur à celui des phases rouge et verte.

5. Registres et opérateurs utilisés

Élément	Rôle
RCC->AHB1ENR	Active l’horloge du périphérique GPIOA sur le bus AHB1.
GPIOA->MODER	Configure le mode des broches. Pour PA5, PA6 et PA7, le champ 01 correspond à une sortie générale.
GPIOA->ODR	Contient les valeurs de sortie appliquées aux broches du port GPIOA.
|=	OU binaire avec affectation : met à 1 les bits ciblés sans changer les autres bits.
&= ~masque	Efface les bits ciblés grâce à un masque inversé.
1U << n	Décale le bit 1 de n positions vers la gauche pour créer un masque.
__NOP()	Instruction sans opération utilisée dans la boucle d’attente.

Point technique important. Les instructions GPIOA->ODR = (1U << n) écrivent la valeur complète du registre ODR. Elles mettent donc à zéro tous les autres bits de sortie de GPIOA, y compris les bits qui ne correspondent pas aux trois LED. Cette écriture est acceptable si le programme contrôle seul ces sorties ; sinon, il faut préserver les autres bits au moyen d’un masquage ou du registre BSRR.

6. Procédure de compilation et d’exécution dans Keil µVision

- Ouvrir le projet prévu pour le STM32F407VG et vérifier que main.c contient le programme ci-dessus.
- Enregistrer les modifications avec Ctrl+S.
- Lancer Project → Build Target ou appuyer sur F7. Examiner Build Output et corriger toute erreur signalée.
- Vérifier que la cible, le périphérique de programmation et l’algorithme Flash correspondent à la carte utilisée.
- Télécharger le programme dans la mémoire Flash avec la commande de chargement configurée dans Keil.
- Démarrer la session Debug si nécessaire, puis lancer l’exécution avec Run (F5).
- Observer l’ordre rouge → jaune → vert → jaune et vérifier le câblage des broches.

Cette liste décrit la procédure à suivre. Le succès de la compilation, du téléchargement et de l’essai matériel doit être établi à partir des messages Keil et de l’observation réelle ; il n’est pas affirmé sans preuve.

7. Résultats attendus et validation

Contrôle	Résultat attendu	Preuve à relever
Compilation	Aucune erreur de compilation.	Résumé Build Output.
Programmation Flash	Téléchargement terminé sans erreur.	Message de fin de chargement.
PA5 — rouge	Sortie sélectionnée lors de la première phase.	Observation de la LED rouge.
PA6 — jaune	Sortie sélectionnée aux deuxième et quatrième phases.	Observation de la LED jaune.
PA7 — verte	Sortie sélectionnée lors de la troisième phase.	Observation de la LED verte.
Boucle	La séquence se répète continuellement.	Observation sur plusieurs cycles.

Les durées exactes ne peuvent pas être déduites des seuls nombres passés à delay(). Il faut les mesurer sur la carte si une durée en secondes est demandée.

8. Temporisation et limites

La fonction delay() est une attente logicielle par comptage. Les valeurs 9 000 000 et 4 000 000 représentent des itérations, pas des secondes ni des millisecondes. La durée effective dépend notamment de la fréquence du cœur, du code machine généré et du niveau d'optimisation choisi dans Keil.

Les phases rouge et verte sont programmées avec un comptage plus élevé que les phases jaunes. Cependant, sans mesure ou calibration, il n'est pas possible d'affirmer leurs durées exactes. Pour obtenir une temporisation plus stable et reproductible, l'utilisation d'un timer matériel ou de SysTick correctement configuré est recommandée.

9. Problèmes possibles et solutions

Symptôme	Cause possible	Vérification / solution
Erreur de compilation	En-tête CMSIS absent, cible incorrecte ou erreur de syntaxe.	Vérifier le composant STM32F407VG, les fichiers CMSIS et la première erreur de Build Output.
Aucune LED ne s'allume	Horloge GPIOA, câblage, polarité ou masse incorrects.	Vérifier RCC, les broches configurées, la résistance série et les connexions.
Mauvaise couleur ou mauvais ordre	Correspondance entre PA5/PA6/PA7 et les LED différente du commentaire.	Suivre chaque fil et vérifier les broches réellement utilisées.
Séquence trop rapide ou lente	Délai logiciel non calibré.	Mesurer le temps réel ou utiliser un timer matériel.
Autres sorties GPIOA à zéro	L'affectation complète du registre ODR remet les autres bits à zéro.	Préserver les autres bits ou utiliser BSRR si nécessaire.

10. Conclusion

Ce TP met en application la commande de trois sorties numériques du STM32F407VG par accès direct aux registres. L'activation de l'horloge de GPIOA via RCC, la configuration des broches PA5, PA6 et PA7 avec MODER, puis l'écriture des niveaux logiques dans ODR permettent de réaliser la séquence de LED définie dans le programme.

La boucle infinie répète le cycle rouge → jaune → vert → jaune. La temporisation reste approximative et doit être mesurée pour déterminer les durées réelles. La validation finale repose sur une compilation sans erreur, un téléchargement réussi et l'observation du montage physique.